

상태:	신규	시작일:	2026/06/25
우선순위:	보통	완료일:	
담당자:	Redmine Admin	진척도:	0%
범주:		추정시간:	0:00 시간
목표버전:		소요시간:	0:00 시간

설명

Claude Code 작업 지시서 — 등급분류 테스트 데이터 생성 · 분류 · KPI 비교 프로그램

이 문서는 **Claude Code**에게 주는 빌드 스펙이다. Claude Code는 이 문서와 함께 전달되는 **data_classifier.py**(C/S/O 등급분류 모델: 정규식 · NER · BERT 티어 + 오피스/HWPX 추출)를 **import** 해서 호출하는 프로그램 일체를 작성 · 실행 · 검증한다. **data_classifier.py** 자체는 수정하지 않는다.

0. 목적

- 1. 테스트 데이터 자동 생성 — TXT · DOCX · HWPX+ PPTX · XLSX 포함, 정답 라벨 동반.
- 2. 생성 데이터를 **data_classifier**로 분류.
- 3. 예측 vs 정답을 비교하고 **KPI**를 산출 · 비교(설정/티어별 비교 포함).

핵심 원칙(반드시 준수): 실제 개인정보 사용 금지(전량 합성), 정답은 "진짜 본질" 기준으로 라벨, 재현 가능(시드 고정), 추출 실패와 분류 실패를 분리 귀속.

1. 의존 대상 — data_classifier.py 인터페이스(계약)

Claude Code는 아래 함수를 그대로 호출한다.

```
from data_classifier import classify, classify_text, extract_text

# 파일 → 등급 결과 dict
classify(file, n2sf_mode=True, verbose=False, llm_mode=False, *,
         model="minilm", weights=None, locale="ko", ensemble_method="escalate") -> dict
# 텍스트 → 등급 결과 dict (추출 생략)
classify_text(text, *, locale="ko", n2sf_mode=True, verbose=False,
              llm_mode=False, model="minilm", weights=None, ensemble_method="escalate") ->
dict
# 파일 → (text, file_type, warnings)
extract_text(file) -> (str, str, list)
```

읽어야 할 반환 키(이름 그대로):

- result["grade"] = "C"|"S"|"O"(n2sf_mode=True), result["gradeFull"] = OPEN|SENSITIVE|CONFIDENTIAL
- result["score"], result["confidence"], result["locale"]
- result["pii"]["totalCount"], result["pii"]["byType"] → 각 {type,label,count,confidence,source,rule,spans, items(verbose)}
- result["keywords"], result["shap"]["contributions"]
- result["ensemble"], result["tiers"]["rules"]|"ner"|"neural" → 각 {grade,confidence,version,...}
- result["compliance"]["violations"], result["stats"]["bulkPii"]|"bulkPiiCount"
- result["file"]["warnings"], (verbose 시) result["file"]["textSnippet"]

등급 순서: O < S < C(랭크 0/1/2). 과소/과분류 판정에 사용.

지원 포맷: docx/docm, **xlsx/xlsm**, **pptx/pptm**, hwp/hwp, txt/csv/md/json/log/tsv, pdf(opt), hwp(opt).

의존성: presidio-analyzer, spacy(+ 로케일 모델). llm_mode=True는 sentence-transformers/transformers 추가 필요(옵션).

2. 만들 프로그램(3부)

- 1. **생성기 gen/** — 합성 문서 + 라벨 생성 (§ 3).
- 2. **러너 run/** — 생성 문서를 classify로 분류, 설정/티어별 실행 (§ 4).
- 3. **평가기 eval/** — 예측 vs 정답 비교, KPI 산출 · 비교 · 게이트 · 리포트 (§ 5).

CLI 한 줄로 전체 파이프라인이 도는 엔트리포인트(main.py 또는 make)를 제공.

3. 테스트 데이터 생성 요구사항

3.1 층화 차원(각 문서에 라벨로 기록)

- 등급 C/S/O
- 동인 **driver**: A(식별자/PII) · B(의미적 기밀, PII 없음) · marking(대외비 표식) · mosaic(조합) · negative(하드네거티브)
- 티어 타겟: regex · ner · bert
- 로케일: ko 기본(필요 시 ja/en/zh)
- 포맷: txt/docx/hwp/pptx/xlsx

3.2 A형 — 식별자 (검증 통과하는가짜, 실제 PII 금지)

- 체크섬 · 구조가 유효한 합성 식별자를 생성:
 - 주민등록번호: **유효 검증숫자**(부록 A) + 합성 생년월일.
 - 사업자등록번호: **유효 검증숫자**(부록 A).
 - 신용카드: **Luhn 유효** 합성 번호(테스트용 BIN).
 - 계좌 · 전화 · 이메일 · 주소 · 이름: Faker(ko_KR) 등. **우연한 실존 PII 방지**(명백한 합성 범위).
- 문서 템플릿(계약서 · 명부 · 양식)에 슬롯 주입.

3.3 B형 — 의미적 기밀 (PII 없이 내용으로 C/S)

- **시드 뱅크**: 전략 · 협상 · M&A · 법무대응 · 보안 · 미공개재무 등 유형별 의미적 C 시드 문장을 **코드/시드 파일로 다수 작성** (Claude Code가 직접 작성 가능).
- **변형 생성**: 시드를 패러프레이즈 · 치환으로 확장(스크립트 내 LLM 호출 사용 가능, **시드는 비민감 합성만**).
- **PII가 들어가지 않도록** 보장(들어가면 A형과 혼동).

3.4 하드 네거티브 (FP · cry-wolf 통제)

- 식별자처럼 보이거나 가짜: 10~16자리 무작위 숫자, **체크섬 불일치** RRN/사업자/카드, 99월 같은 **구조 위반** 숫자.
- 키워드 부분문자열: "secretary"(→"secret"), "privatewealth"(→"private") 등.
- **인명 다수 비명부**: 이름은 많지만 명부가 아닌 일반 문서(PERSON 대량 오탐 유도).
- 부정 · 범위 문맥: "기밀 아님", "공개용", "대외비 해제" 등.

※ 정답 라벨은 **본질 기준**: 체크섬 불일치 숫자의 true grade는 **O**(가짜이므로). 현재 분류기는 RRN 체크섬이 없어 이를 C로 오탐할 수 있는데, **그 오탐이 KPI에 드러나도록** true=O로 라벨한다. (라벨을 "정규식이 잡을 법한 것"으로 맞추지 말 것.)

3.5 포맷 주입 —PPTX · XLSX 포함, 포맷별 취약 위치에 심기

같은 텍스트 페이로드를 각 포맷의 **누락 위험 위치**에 배치한다.

포맷	라이브러리	주입 위치(필수)
XLSX	openpyxl	숫자 셀 (계좌 · 전화 · 사번을 숫자형으로), 공유문자열 vs inline string 혼합, 여러 시트
PPTX	python-pptx	발표자 노트 , 표 셀, 일반 텍스트박스
DOCX	python-docx	머리/꼬리말 , 각주/미주, 주석, 본문
HWPX	직접 zip 작성	여러 Contents/section*.xml, header
TXT	-	평문

- 각 문서에 **expected_extraction_strings**(추출 텍스트에 반드시 살아남아야 할 리터럴 목록)를 라벨로 기록 → 추출 생존율 측정용. **XLSX** 숫자 PII 생존이 핵심.

3.6 정답 라벨 스키마 (labels.jsonl, 문서당 1줄)

```
{
  "id": "doc_000123",
  "path": "data/files/doc_000123.xlsx",
  "format": "xlsx",
  "locale": "ko",
  "driver": "A",
  "tier_target": "regex",
  "grade_true": "C",
  "entities_true": [
    { "type": "KR_RRN", "value": "<합성값>", "valid": true, "site": "xlsx_numeric_cell" }
  ],
  "expected_extraction_strings": [ "<합성값>", "..."],
  "notes": "직원명부 35행"
}
```

- 합성 데이터이므로 **구성 시점에 자동 라벨**. B형 의미 등급은 시드별로 grade를 명시.
- 데이터셋 매니페스트(manifest.json): 층화별 건수 집계.

3.7 규모(설정 가능, 기본 권장)

- 스모크 100 / 골드 1,0002,000(그중 ~~**B형~~ 150300 별도**) / 운영분포 5001,000 / 적대 100300 / 추출 픽스처 포맷당 20~50.
- 재현성: 모든 무작위에 **단일 RNG 시드**(config). 재실행 시 동일 데이터셋.

4. 분류 실행(러너)

- 각 파일을 classify(path, n2sf_mode=True, verbose=True, locale=<label.locale>, ...)로 분류.
- ****설정/티어 비교(ablation)****를 위해 다음 조합을 **각각 실행**(config로 on/off):
 - llm_mode = False(규칙+NER) / True(+BERT, 모델 선택)
 - ensemble_method ∈ {escalate, vote, weighted, max-rank, soft}
 - (옵션) locale, model 변형
- 각 실행 결과 dict를 원문 그대로 runs/<config>/predictions.jsonl에 저장(추후 재집계용).
- **추출 검증도** 별도로: extract_text(path) 호출 → expected_extraction_strings 생존 여부 기록.
- **결정성 체크**: 동일 설정 2회 실행이 동일 결과인지(해시) 확인(규칙 · NER 결정적이어야 함).

5. 평가 · KPI(평가기)

5.1 분류 KPI (문서 등급)

- 3×3 혼동행렬(true × pred).
- 등급별 recall / precision / F1. C-recall = 1차 KPI.
- macro-F1.
- 과소분류율 = P(pred 랭크 < true 랭크), 특히 P(true=C, pred∈{O,S})(보안 치명).
- 과분류율 = P(pred 랭크 > true 랭크)(cry-wolf 지표).
- 기권율(해당 시).
- Wilson 95% 신뢰구간을 recall에 병기(유한 N).

5.2 스트라타별 분해(필수)

- driver(A/B/marking/mosaic/negative) · locale · format · tier_target별로 위 지표 슬라이스.
- 의미적 C(B형) 슬라이스와 negative 슬라이스를 명시적으로 별도 보고(전자는 recall, 후자는 FP).

5.3 설정/티어 비교(ablation 표)

- llm_mode off/on 및 ensemble_method별로 C-recall · macro-F1 · 과분류율을 나란히 표로.
- 또한 각 예측의 result["tiers"]를 읽어 **티어별 등급 분포**도 보고(어느 티어가 상향을 일으켰는지).

5.4 추출 생존율(포맷별)

- 포맷별 **추출 재현율** = expected_extraction_strings 중 추출 텍스트에 살아남은 비율.
- **XLSX 숫자 PII 생존율**을 별도 지표로.

5.5 캘리브레이션 · 결정성

- confidence vs 실제 정확도(신뢰도 구간별), **ECE**.
- 결정성 위반 건수(있으면 0이어야).

5.6 취합 기준(게이트)

- **C-recall**을 합격/불합격 **게이트**로: $C_recall \geq sla.c_recall(config, 기본값 예: 0.95)$ 미달이면 **FAIL** 반환(종료코드 $\neq 0$) — CI 게이트.
- 단일 정확도는 보조로만. 항상 **스트라타 · CI와 함께** 보고.
- **귀속 분리**: true 엔티티가 추출 텍스트에 없으면 "추출 실패", 있는데 분류기가 등급에 반영 못 하면 "분류 실패" 로 FN 원인을 나눠 집계.

6. 산출물(파일 트리) · 출력 포맷

```
project/
  config.yaml          # 시드 · 규모 · 포맷 · 로케일 · sla 게이트 · classify 설정
  main.py              # 전체 파이프라인 엔트리포인트
  gen/                 # 식별자(체크섬) · B형 시드 · 하드네거 · 포맷주입 · 증강 · 라벨
작성
  run/                 # classify 호출, 설정별 실행, 추출검증, 결정성
  eval/                # 지표 · 스트라타 · ablation · 추출생존 · 캘리브 · 게이트 · 리포트
  tests/              # 생성기 자체 테스트(체크섬 유효 assert, 주입 생존 asse
rt)
  data/
    files/             # 생성된 문서들
    labels.jsonl       # 정답 라벨
    manifest.json      # 증화별 건수
  runs/<config>/predictions.jsonl
  reports/
    report.md          # 사람이 읽는 리포트(표 · 게이트 결과)
    metrics.json       # 기계 판독 지표(전 스트라타)
    confusion_<config>.csv
```

- 콘솔 요약 + report.md(혼동행렬, 등급별 PR, C-recall 게이트 PASS/FAIL, 스트라타 · ablation 표, 추출 생존율) + metrics.json.

7. 환경 셋업(러너가 수행)

- pip install presidio-analyzer spacy faker python-docx python-pptx openpyxl numpy 등.
- python -m spacy download ko_core_news_sm(필요 로케일 모델).
- llm_mode 테스트 시 pip install sentence-transformers transformers torch(옵션). 미설치면 규칙+NER만으로 진행하고 리포트에 명시.
- 네트워크/모델 다운로드 실패해도 **규칙 · NER 경로로 평가가 끝까지 돌도록** 방어.

8. 가드레일(반드시 준수)

1. **실제 PII 절대 금지** — 전량 합성. 식별자는 검증 통과하되 실존 인물과 무관.
2. **정답은 본질 기준** — 가짜 식별자/체크섬 불일치는 true=O(또는 의도 등급). "정규식이 잡을 법한가"로 라벨하지 말 것.
3. **재현성** — 단일 시드로 데이터 · 실행 재현.
4. **귀속 분리** — 추출 실패 vs 분류 실패.
5. **분류기 무수정** — data_classifier.py는 import만.
6. **B형 생성 독립성** — 시드는 비민감, (LLM 변형 사용 시) 평가 베이스라인과 모델 계열 혼선 주의.

9. 수용 기준(완료 정의)

- python main.py --config config.yaml 한 번으로 **생성** → **분류** → **평가** → **리포트**가 끝까지 돈다.
- data/에 라벨 · 매니페스트와 **PPTX · XLSX 포함** 전 포맷 문서가 생성된다.
- tests/가 통과한다(생성 식별자 체크섬 유효, 주입 문자열 추출 생존).
- reports/report.md에 **C-recall 게이트 결과** + **스트라타별 · ablation 표** + **포맷별 추출 생존율**이 출력된다.
- C-recall 게이트 미달 시 **비정상 종료코드**로 CI가 실패하도록 한다.

부록 A. 검증숫자 알고리즘(생성 · 하드네거 모두에 사용)

- **주민등록번호(13자리)**: 가중치 $w=[2,3,4,5,6,7,8,9,2,3,4,5]$ 를 앞 12자리에 곱해 합 S . 검증숫자 $c=(11-(S\%11))\%10$. 13번째 자리 = c . (하드네거는 c 가 아닌 값으로.)
- **사업자등록번호(10자리)**: 가중치 $w=[1,3,7,1,3,7,1,3,5]$ 를 앞 9자리에 곱해 합 S , $S += (d9*5)//10$. 검증숫자 $c=(10-(S\%10))\%10$. 10번째 자리 = c .
- **신용카드(Luhn)**: 오른쪽부터 짝수 위치 숫자를 2배(>9 면 -9), 전체 합이 $\%10==0$ 이면 유효.
- **구조 위반 하드네거 예**: RRN 앞 6자리 월=13 · 일=45 등 불가능 값.

생성기는 위 공식으로 **유효 식별자**와 **불일치 하드네거티브**를 둘 다 만들 수 있어야 한다.